

# Mizan.ai — PDF Editor Feature

Scoping Handoff (Final) | Companion to the DOCX/XLSX → PDF format converter

**Status:** Planning/scoping document only — not a request to implement yet. Hold until the cold-start / GPU-snapshot work is complete.

## Why this feature

Users currently have to leave Mizan.ai to do basic PDF editing (annotate, fill forms, reorder pages) on documents they've already touched via the format converter. Goal is a “no need to leave the app” convenience layer — **not** an attempt to compete with full PDF editors like Adobe Acrobat. Scope is deliberately narrow to avoid becoming a generic utility that dilutes the ZATCA-compliance differentiator. Conversion alone doesn't move the needle much; pairing it with these basic editing options makes Mizan feel like a real document workspace rather than a one-trick converter.

## Guiding principle

Set explicit boundaries in the UI up front (e.g. “Edit annotations, fill forms, and reorder pages — not full text editing”), same philosophy as labeling PDF-to-DOCX as “coming soon” rather than pretending to support everything.

## Tier 1 — Build this for first release (easy, days not weeks)

These only overlay/manipulate the PDF, never touch existing embedded text content. All well-supported by standard libraries (PyMuPDF / pypdf on the backend, pdf-lib or pdf.js on the frontend for interactive editing).

- **Highlighting / annotations** — highlight text, add sticky-note style comments, draw shapes/freehand marks on top of a page.
- **Text boxes** — let the user drop a new text box anywhere on a page (e.g. adding a note, a stamp-like label). This is an overlay, not an edit of existing text.
- **Form filling** — fill in existing AcroForm fields (checkboxes, text fields, dropdowns) if the PDF has them. Very common for structured invoice/compliance templates.
- **Signatures** — draw or upload a signature image and place it on a page.
- **Page reordering** — drag-and-drop reorder of pages within a single PDF, and delete individual pages.
- **Watermarking / stamping** — stamp a logo, “draft”, or similar label across one or all pages. Pure overlay, same effort class as a text box.
- **Page rotation** — rotate individual pages or the whole document. Single-operation, trivial with the same libraries already in use.

*Suggested stack:* PyMuPDF (fitz) on the backend for the actual PDF manipulation, pdf.js for rendering + a lightweight canvas overlay on the frontend for the interactive annotation/placement UI.

## Quick win pulled forward from Tier 2

- **Password protection** — add an open password or permissions password (e.g. restrict printing/copying) to a PDF. Technically medium-tier since it touches document security/encryption rather than just visuals, but most PDF libraries have this built in, so it's realistically a day of work. Worth including as a “good to have” alongside the Tier 1 list rather than parking it.

## Tier 2 — Parked for later (medium effort, 1–2 weeks)

Explicitly NOT in scope for the first pass. Listed here so the boundary is clear and intentional, not an oversight.

- **Merge PDFs** — combine multiple PDFs into one file. Needs a page-picker/reorder UI across multiple source files.
- **Split PDFs** — extract a page range into a new file.
- **Redaction** — properly remove (not just visually cover) sensitive text so it can't be copy-pasted out afterward. Requires actually deleting the underlying text/content stream, not just drawing a black box on top — this is what makes it medium rather than easy.
- **Delete a line item from an invoice table** — requires reflowing the rest of the table (other rows shift up) and potentially recalculating totals. Touches document structure and logic, not just visuals.
- **Compress a PDF** — reduce file size, useful for large scanned invoice uploads. Fairly quick (roughly half a day) but grouped here rather than pulled forward, since it's not something users have been asking for directly — revisit if scanned-file-size complaints come up.

## Explicitly out of scope (very hard / not practical solo)

- True in-place text editing of existing PDF content (e.g. clicking on an existing amount or company name in the PDF and changing it), beyond a narrowly scoped invoice-template case. PDFs store characters as positioned glyphs, not structured paragraphs, so this requires font and layout reconstruction — a multi-week-to-multi-month effort even for experienced teams if attempted generically.
- If ever pursued, scope strictly to simple, predictable invoice-style layouts (leveraging the existing ZATCA knowledge base to identify editable fields) rather than arbitrary PDF layouts.

## Sequencing note

This scoping doc was written in parallel while cold-start/GPU-snapshot work is in progress on a separate thread. Do **not** bundle implementation of this feature into the same handoff/PR as the format-conversion work (Feature H) — keep them as separate, independently testable tasks. Pick this up only once cold-start work is finished and format conversion has shipped.